

Design the Interactions First

JOAT Labs, pm-substrate, and the Project-Manager Layer for Agentic Workspaces —
Expanded First-Principles Draft

Emmanuel Akinwale, Founder, JOAT Labs

May 26, 2026

Executive Thesis

Modern businesses do not suffer from a shortage of software. They suffer from a shortage of coordination. The average organization already has specialized systems for sales, operations, finance, documents, communications, analytics, and delivery. Each tool may be competent inside its own boundary. The failure appears between those boundaries: state is copied, context is lost, ownership is unclear, and every new tool or AI feature adds another place where work can drift out of sync.

JOAT Labs is building for that gap. The company is not trying to add one more isolated application to the workspace. It is building the project-manager layer for the workspace: the layer that keeps tools, people, workflows, and agents aligned around shared operational state. In software terms, that layer is **pm-substrate**: a tenant-scoped operational substrate made of actors, entities, relationships, events, capabilities, policies, workflow state, evidence, projections, commitments, and continuity records.

The central claim is simple: if the value of a system comes from the interaction of its parts, then the modern workspace must be designed around interactions first.

In the AI era, this becomes urgent. Agents can call tools, but tool access is not operational memory. A model call is not a system of record. A prompt is not a workspace. Agents need durable state, causality, permissions, workflow position, freshness, and a reliable model of what changed. Without that substrate, agents inherit the same fragmentation that already makes SaaS work expensive and brittle.

The deeper claim is stronger:

The AI state problem is a special case of state coherence under partial observation and change. An agent is a bounded perception-action system. Because every bounded agent maintains only a local model of a larger changing environment, coordination fails unless local models are reconciled against an authoritative shared substrate. pm-substrate is that substrate for work.

The Problem Is Not More Tools

Most business software has been built as if the unit of value were the individual tool: better CRM, better project tracker, better calendar, better inbox, better accounting system. That orientation made sense when the market lacked capable components. It makes less sense now. Modern companies are surrounded by competent components, yet their operating reality is still fragmented.

The market is not telling us that businesses need another disconnected surface. It is telling us that the surfaces no longer compose. Application switching is not merely a distraction problem. It is a symptom of work being distributed across systems that do not share enough context to behave like one environment.

This is the workspace problem JOAT Labs is organized around. Businesses do not need a new shelf. They need the shelf to become coherent.

The Hidden Bottleneck Is State

The word integration makes the problem sound narrower than it is. Integration suggests that Tool A needs to send data to Tool B. That is necessary, but insufficient. The harder question is whether the workspace has a reliable shared understanding of what the data means, who owns it, when it changed, which version is current, what process it belongs to, and what actions are allowed next.

That is a state problem.

State is not just a database row. State is the current shape of the work: the customer and the contract, the project and its deadlines, the vendor and its obligations, the invoice and its approval status, the message and the event it caused, the workflow and the step it is waiting on. When state is scattered across apps, every tool builds a partial reality. Humans then become the integration layer, manually carrying context from one surface to another.

A workspace without shared state produces three predictable costs:

1. **Context cost** — people and systems rediscover what is already known somewhere else.
2. **Coordination cost** — handoffs require meetings, messages, reminders, and manual reconciliation.
3. **Trust cost** — when two systems disagree, no one knows which one represents reality.

These are not edge cases. They are the daily texture of fragmented work.

What an Agent Is, Structurally

An agent is not first a chatbot, a model, or an automation. At the lowest useful level, an agent is a bounded system coupled to an environment through perception, internal state, action, and feedback.

More formally:

1. There is an environment with some state: S .
2. The agent receives partial observations from that environment: O .
3. The agent maintains an internal representation or belief: M .
4. The agent updates that representation as observations arrive: $M_{t+1} = f(M_t, O_t)$.
5. The agent selects an action according to a policy, objective, or decision function: $A_t = \pi(M_t, G_t)$.
6. The action changes the environment or the agent's future observations: $S_{t+1} = T(S_t, A_t)$.
7. The agent receives feedback, error, reward, or evidence about the result.

This structure exists before artificial intelligence. It describes animals, people, departments, companies, robots, software agents, and LLM-based systems. The common structure is the same:

Agent = stateful perception-action loop.

In system language:

An agent is an open, bounded subsystem that maintains an internal model of an external system and uses that model to choose actions that change the external system.

This means the state problem is not an implementation detail added by LLMs. It is present at the root of agency. A bounded agent never has direct access to total reality. It acts from partial observations, compressed representations, uncertain beliefs, incomplete goals, and delayed feedback.

The First-Principles Ladder

The agent-state problem can be decomposed below AI:

1. **Distinction / variable** — something can be in one condition rather than another.
2. **State space** — the set of possible configurations.
3. **Relation / graph** — variables are connected; objects are not isolated.
4. **Information** — observations reduce uncertainty; memory compresses information over time.

5. **Linear algebra / representation** — state can be encoded as vectors, matrices, projections, embeddings, and feature spaces.
6. **Probability / statistics** — the agent infers hidden state from noisy observations.
7. **Dynamics / transition** — state evolves through time and action.
8. **Control / feedback** — the agent acts, observes the result, and adjusts.
9. **Computation / model** — the model maps inputs to outputs.
10. **Agency / policy** — the system chooses actions under goals or objectives.
11. **Multi-agent coordination** — many bounded actors maintain local models and act concurrently.
12. **Institutional governance** — organizations define authority, permission, accountability, retention, and audit.

The current AI conversation usually starts too late, at memory, tools, vector databases, or context windows. Those are real issues, but they sit inside a deeper structure: state coherence under partial observation and change.

The Nested State Problem

Once the agent is decomposed, the state problem appears at every layer.

Agent layer	State question	Failure mode
Environment	What is actually true?	Reality is distributed across systems and actors.
Observation	What can the agent see?	Inputs are partial, stale, noisy, missing, or misformatted.
Representation	How are observations encoded?	Compression loses structure, authority, or causal meaning.
Context	What is currently in working memory?	Context overflow, missing facts, prompt contamination.
Memory	What persists over time?	Forgetting, over-remembering, bad summaries, stale facts.
World model / belief	What does the agent think is true?	Beliefs become incomplete, contradictory, ungrounded, or outdated.
Goal / objective	What is the agent trying to optimize?	Goals are ambiguous, conflicting, underspecified, or changed midstream.
Plan / policy	What should happen next?	Plans become invalid because state changed.
Actuators / tools	How does the agent change the world?	Tools have side effects, authority limits, and hidden downstream consequences.
Feedback / evaluation	Did the action work?	Feedback is delayed, missing, misleading, or disconnected from the original action.
Identity / continuity	Is this the same accountable actor over time?	No durable responsibility, compounding learning, or trust.

An LLM agent inherits all of these problems. The model weights provide statistical memory, but not situation-specific operational state. The context window provides working memory, but it is finite and fragile. Retrieval provides recall, but not authority. Tool outputs provide observations, but only from each tool’s perspective. Plans provide temporary structure, but not necessarily valid workflow state. Actions mutate the world, but without causal tracking the agent may not know what actually changed. Feedback may arrive late or not at all. Identity is often missing entirely.

Therefore the AI state problem is not one problem. It is a stack of state problems nested inside an agent system.

A State Taxonomy

The word state is often overloaded. The thesis must separate its meanings:

1. **Reality state** — what is actually true in the environment.
2. **Observed state** — what an actor can see.
3. **Represented state** — how observations are encoded.
4. **Belief state** — what the actor thinks is true.
5. **Operational state** — the current actionable configuration of work.
6. **Authoritative state** — the state the institution accepts as binding.
7. **Historical state** — the event sequence that explains how current state came to be.
8. **Projected state** — derived views built from authoritative records.

pm-substrate does not govern all of reality. It governs operational, authoritative, historical, and projected workspace state. This distinction matters. The substrate is not the agent's complete world model. It is a **governed operational model of the workspace**.

Pass 1 — Fragmented Tools and Local Models

The first pass is the visible workspace failure: every actor works from a local model.

A sales tool has one model of the customer. A project tracker has another model of the work. A calendar has another model of time. A finance system has another model of payment and obligation. A human has a memory of the conversation. An agent has a prompt, retrieved snippets, and tool outputs.

None of these local models is the whole workspace.

The failure mode is not simply missing integration. It is model divergence. Each actor acts as if its local representation is sufficient. Over time, the organization pays in rework, meetings, repeated explanations, stale handoffs, contradictory records, and low trust.

Pass 1 claim:

The modern workspace is a multi-agent system whose agents act from divergent local state.

Pass 2 — Operational State, Not Transcript

The second pass corrects the common AI mistake: state is not chat history.

A transcript records what was said. Operational state records what is currently true enough to act on. A memory summary can be useful, but it is not automatically authoritative. An agent can remember a PR as open after GitHub has merged it. A user can say a callback URL is configured, invalidating a previous plan. A deployment can succeed after the last written note said it was blocked.

Operational state must therefore include provenance and freshness:

- source;
- timestamp;
- authority;
- evidence;
- validity window;
- dependencies;
- invalidation condition;
- downstream effects.

Pass 2 claim:

The agent state problem is not forgetting. It is the absence of structured, evidence-backed, freshness-aware operational state.

Pass 3 — Coordination State as Contract

The third pass moves from individual state to shared coordination.

Once multiple humans, tools, workflows, and agents interact, state becomes a contract. The system must know which source is authoritative for which fact, which capability may mutate which object, which workflow transition is valid, and which actor is accountable.

This is where pm-substrate diverges from generic memory systems. A vector store can retrieve relevant text. It cannot decide that GitHub is authoritative for PR state, Cloudflare is authoritative for deployment state, Stripe is authoritative for payment state, and the substrate event log is authoritative for workflow transition history.

Pass 3 claim:

Shared state is not a pile of remembered context. It is a coordination contract among bounded agents.

Pass 4 — Temporal and Causal State

The fourth pass makes time first-class.

State is not a snapshot. It is the result of events. Every important workspace fact should have a creating event, a causal chain, a freshness horizon, dependencies, and an invalidation path.

This matters because agents act across time. A plan created yesterday can become unsafe today. A tool output from ten minutes ago can expire. A reminder can fire after the need has already been handled. A workflow step can be blocked by an upstream event that the agent never observed.

pm-substrate needs event sourcing not because logs are fashionable, but because causality is the backbone of trustworthy action. The event log answers:

- what happened;
- who or what caused it;
- which entity it affected;
- which workflow it advanced;
- what evidence supported it;
- what projections changed;
- what should be invalidated or recomputed.

Pass 4 claim:

An agent cannot coordinate safely unless state is temporal, causal, and invalidatable.

Pass 5 — Institutional State Governance

The fifth pass is the deepest layer.

Humans do not run organizations from memory. They externalize state into contracts, calendars, issue trackers, ledgers, permissions, dashboards, rituals, escalation paths, and audit trails. Those artifacts exist because no individual sees the whole organization.

Project management is a human solution to distributed agent state. The project manager maintains shared state, current goals, unresolved blockers, handoff boundaries, accountability, next actions, decision memory, and feedback loops.

pm-substrate formalizes the operational-state function of project management for workspaces where the agents include humans, software tools, automations, and LLM workers.

It does not replace the full human role of a project manager. It does not replace persuasion, prioritization, conflict mediation, stakeholder judgment, or value tradeoffs. It formalizes the state substrate that makes those human functions less brittle.

Pass 5 claim:

The endpoint is not agent memory. The endpoint is institutional state governance for bounded agents acting over a shared changing reality.

What pm-substrate Is

pm-substrate is the implementation of the JOAT Labs thesis. It is not a single vertical application and it is not a point-to-point integration script. It is a shared operational substrate that lets tools and agents coordinate through declared contracts instead of private assumptions.

Its minimal primitive set is:

Primitive	Role
Actor	Human, tool, agent, organization unit, or service principal.
Entity	Durable object of work.
Relationship	Typed link between entities.
Event	Immutable state-transition record.
Capability	Declared action interface.
Policy	Permission or validation rule.
Workflow state	Current process position.
Evidence	Source-backed observation.
Projection	Derived operational view.
Commitment	Promised future action or obligation.
Continuity record	Resumable decision/context checkpoint.

These primitives map directly to the agent-state stack:

Agent-state need	pm-substrate primitive
Environment state	Entity graph
State changes	Append-only event log
Possible actions	Capability registry
Valid process	Workflow runtime
Allowed actions	Permissions and validation
Observed facts	Projections with source and freshness metadata
Agent continuity	Continuity ledger
Evidence	Provenance links
Feedback	Workflow outcomes and emitted events
Identity	Tenant-scoped human, tool, and agent principals

The central design law is:

No agent's local state is authoritative unless it is reconciled with substrate state.

Agent memory can propose. Agent belief can explain. Agent plans can recommend. But authoritative project state changes only through substrate transitions.

Authority Promotion Rules

If the substrate is authoritative, the thesis must explain how something becomes authoritative.

A useful promotion ladder is:

1. **Observation** — a human, tool, or agent observes something.
2. **Evidence** — the observation is recorded with source, time, actor, and provenance.
3. **Proposed fact** — the system interprets the evidence as a candidate statement about an entity or workflow.

4. **Validated fact** — schema, permission, freshness, and conflict checks pass.
5. **Authoritative transition** — an event is appended to the log.
6. **Projection update** — derived views and workflow states are recomputed.
7. **Continuity update** — unresolved work, decisions, and evidence are preserved for future agents.

Without this ladder, “reconcile with substrate” is only a slogan. With it, pm-substrate becomes a controlled pathway from observation to operational reality.

A Concrete Flow

Consider a wedding-services business. A vendor contract is signed. In a traditional SaaS stack, that fact may live in a contracts tool while the calendar, planner, budget, and communications systems each need to be updated separately. Humans often become the bridge.

In pm-substrate, the flow is represented as stateful events and capability invocations:

1. The contracts capability emits a typed event: `wedding.contract.signed`.
2. The event log records the event with tenant, entity, causation, and provenance metadata.
3. The calendar capability subscribes to the event and confirms the vendor deliverable window.
4. The planner capability unlocks downstream tasks linked to that contract and deliverable.
5. The budget capability updates projected and actual spend through the graph relationship between contract, vendor, and budget category.
6. The communications capability schedules a reminder to the couple and vendor 72 hours before the deliverable.
7. The continuity ledger preserves what the agent or workflow concluded, what evidence supported it, and what remains unresolved.

No single tool owns the whole outcome. The outcome emerges because the tools coordinate through substrate state.

Why This Is Not Just iPaaS or Workflow Automation

It is natural to compare pm-substrate to iPaaS platforms, workflow automation tools, or API orchestration products. Those systems are useful, but they usually begin from connections: when this happens in one app, do that in another app.

pm-substrate begins from shared operational state.

Point-to-point automation can move data, but it often does not answer deeper questions:

- Which entity is this really about?
- Which system owns this fact?
- Is the downstream action still valid?
- What changed since the trigger fired?
- Which permission boundary applies?
- What happens when a second tool acts concurrently?
- How do we replay the history?
- How does an agent know which memory is evidence and which memory is stale?

pm-substrate treats tools as capability providers over a common graph and event history. It treats workflows as stateful processes. It treats permissions and schemas as runtime gates. It treats agent memory as evidence-backed continuity, not retrieved text.

That is the difference between connecting apps and coordinating work.

The Blockchain Sidebar: State Problem or State Solution?

Blockchain is best understood as a partial state-solution pattern, not a complete answer to the workspace state problem.

A blockchain solves one hard piece: **shared agreement over an ordered history of state transitions among mutually distrustful actors**. It gives participants a replicated ledger, consensus over transaction order, tamper-evident history, and deterministic state transition rules.

In that sense, blockchain is a state solution approach. It asks:

How can a distributed network agree on one canonical state history without trusting a central operator?

But blockchain also has its own state issues:

1. **Canonical state is expensive** — every globally authoritative transition has cost, latency, and throughput limits.
2. **On-chain state is not reality** — it is only the state produced by accepted transactions. The oracle problem remains: how does the chain know what happened off-chain?
3. **Finality is probabilistic or delayed** — different chains provide different settlement guarantees.
4. **State bloat accumulates** — full history and current state grow over time.
5. **Smart contracts are rigid** — code can enforce transitions, but business exceptions often require judgment.
6. **Governance is externalized** — protocol rules do not eliminate human governance; they relocate it to validators, token holders, foundations, forks, and social consensus.

For JOAT Labs, the lesson is not “put pm-substrate on-chain.” That would usually be wrong. Most workspace coordination needs privacy, low latency, rich permissions, reversible corrections, and domain-specific governance.

The lesson is narrower and useful:

- ordered event history matters;
- deterministic transition rules matter;
- replayability matters;
- append-only provenance matters;
- authority over state transitions matters.

pm-substrate should borrow the state-machine discipline of blockchains without inheriting the global consensus machinery unless a domain specifically requires distrust-minimized public settlement.

Falsifiable Claim

A serious thesis needs a test.

Falsifiable version:

If pm-substrate is doing real work, an agent should be able to resume a cross-tool workflow after context loss using only substrate state and complete it with lower stale-action rate, lower rework, and higher evidence coverage than an agent using chat history plus raw tool access.

Candidate metrics:

- state disagreement rate;
- stale action rate;
- contradiction rate;
- evidence coverage;
- handoff latency;
- workflow completion rate;
- rework rate;
- agent resume success without chat history;
- unauthorized action block rate;
- cross-tool outcome success rate;
- unexplained transition rate.

The north star:

Can many bounded agents act from one coherent operating reality?

Conclusion: Design the Interactions First

The next era of workspace software will not be won by adding isolated AI features to isolated tools. That path multiplies fragmented state. The next era will be won by systems that make tools and agents interoperable at the level where work actually happens: entities, events, permissions, workflows, evidence, commitments, and continuity.

The workspace should not treat state as something hidden inside applications. State is the shared operating reality through which work advances.

The agent should not be treated as a magic worker with memory. An agent is a bounded perception-action system operating under partial observation. It needs a governed operational substrate to act safely over time.

That is the role of pm-substrate. It is the project-manager layer for agentic workspaces: the system that keeps bounded agents aligned around shared state while preserving authority, causality, permission, and accountability.

The principle is simple:

Design the interactions first.

References and Source Threads

This expanded draft preserves the source threads from the previous thesis draft and adds first-principles framing from systems, control, distributed systems, databases, information theory, and agent architecture. Citations to verify or expand in the next evidence pass include:

- General systems theory: Bertalanffy.
- Systems thinking and coordination: Ackoff, Meadows, Forrester, Senge.
- Sociotechnical systems: Trist and Bamforth.
- Systems-of-systems interoperability: Fisher / SEI.
- State estimation and control: Kalman filtering, observers, cybernetics, Ashby's law of requisite variety.
- Distributed systems: state machine replication, event logs, consensus, idempotency.
- Databases: transactions, constraints, isolation, append-only logs.
- LLM agents: surveys on profile, memory, planning, action, tool use, and multi-agent workflows.
- State-driven agent workflows: StateFlow.
- Enterprise information systems: composable architecture and executable business processes.
- HCI: task-centric application switching and cross-application work.
- SaaS fragmentation: BetterCloud, Zylo, MuleSoft.